

ML for Network Traffic Anomaly Detection

Arpan Kapoor

COLLEGE OF ENGINEERING

UNIVERSITY *of* WASHINGTON

Problem Description

Goal: detect malicious network traffic from within large volumes of benign traffic.

- Cyber attacks compromise sensitive data, disrupt infrastructure, and cause massive financial damage.
- They affect organizations maintaining the network and users relying on the organizations' services.



Attack Model

Types of attacks include: **DDoS**, **Bot**, **PortScan**, **SQL Injection**, **Web Attacks**, etc.



- These attacks manipulate network attributes like packet lengths, window sizes, and subflow bytes
- Often designed to mimic normal traffic
- Normal traffic regularly changes, so static defensive measures are inadequate

Why Machine Learning?

Rule-based or signature-based detection systems:

- Snort or Suricata
- Zero day attacks: can only catch known attacks
- Fail to adapt to shifting network traffic.

Machine learning generalizes patterns in the data:

- Supervised models can learn complex attack patterns
- Unsupervised models can learn the geometric boundary of normal traffic to flag anomalies.



Dataset and Data Characteristics

Intrusion Detection Evaluation Dataset CIC-IDS2017 (GeneratedLabelledFlows ver.)



- Each row represents a summarized network flow connection
- Primarily numerical/continuous features
 - min_packet_length, packet_length_std, ...
- Over 80 features with around 2.3 million rows
- Somewhat imbalanced: 76% benign, 24% malicious



Data Preprocessing and Feature Engineering

Preprocessing:

- Converted attack type label into binary malicious flag
- Cleaned column names and dropped any Inf/NaN values
- Converted timestamps to datetime values

Feature Extraction:

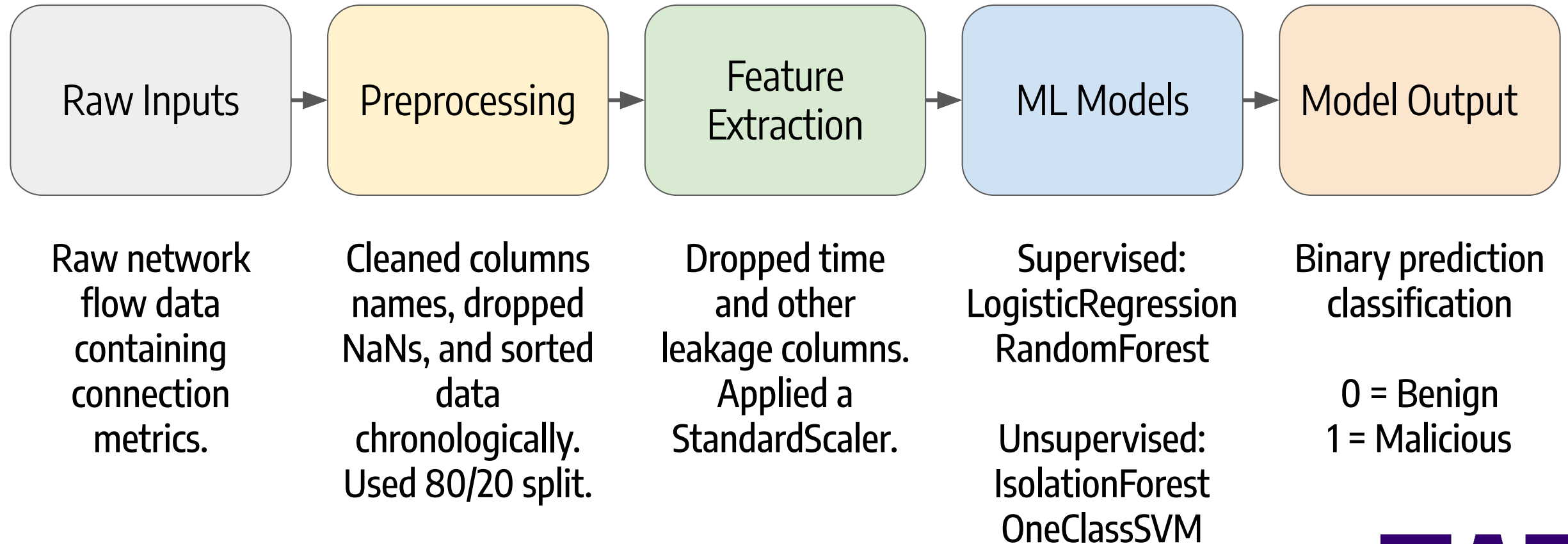
- Dropped leakage columns and any non-numeric columns
- Dropped time after sorting chronologically

Scaling:

- Applied StandardScaler for use in LogisticRegression and OneClassSVM



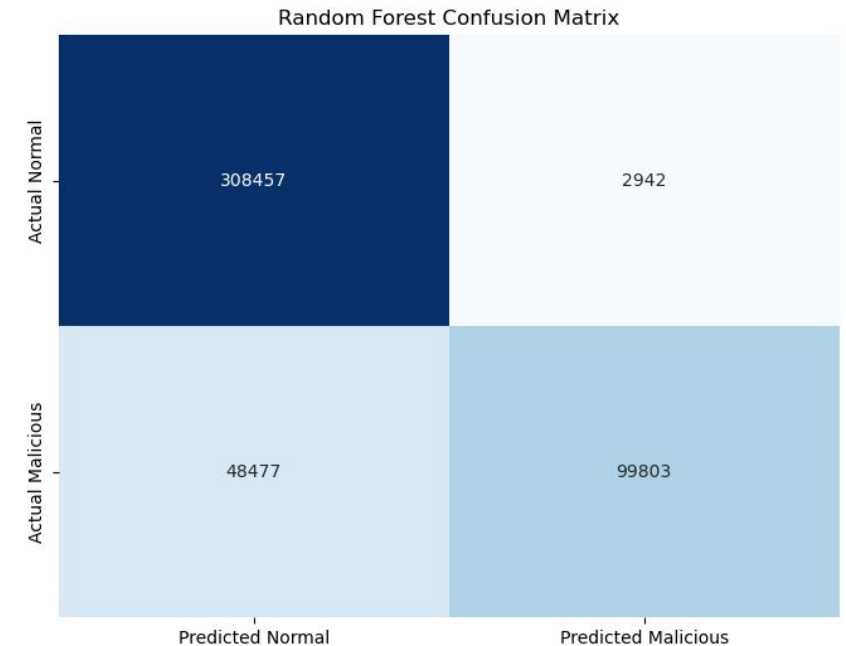
Machine Learning Pipeline



Evaluation Methodology

Focused on recall at a 1% & 5% false positive rate (FPR):

- Accuracy can be misleading because of class imbalance
- Important to minimize false positives and false negatives when detecting attacks
- Maximize threat detection (recall) without exceeding 1% noise limit
- Used a chronological split to test how models handle data drift



Results

RandomForest achieved 67.307% recall at a forced 1% FPR:

```
RandomForestClassifier Results at 1.0% FPR:
```

```
Actual FPR: 0.945%
```

```
Recall: 67.307%
```

```
Threshold: 16.000%
```

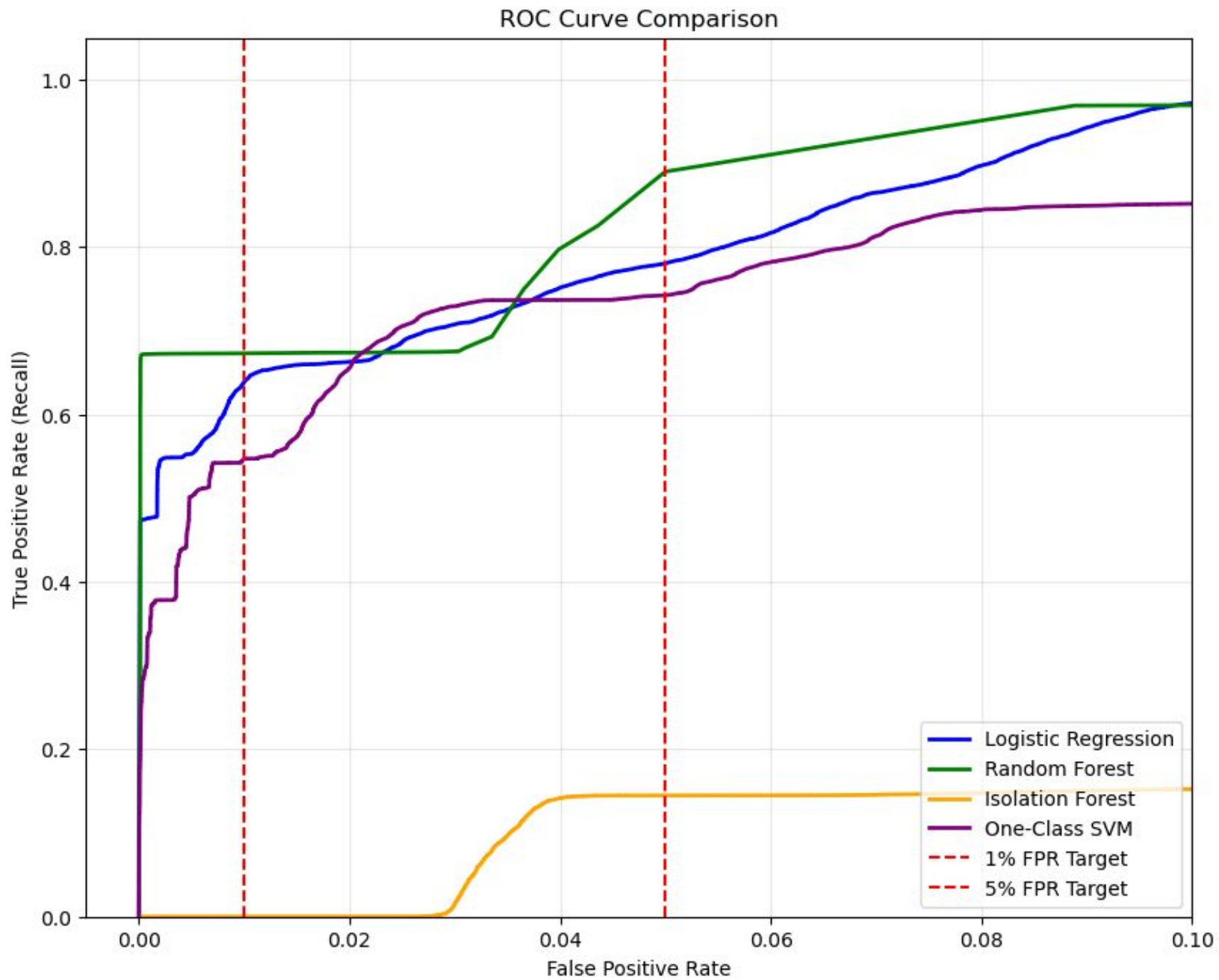
```
RandomForestClassifier Results at 5.0% FPR:
```

```
Actual FPR: 4.998%
```

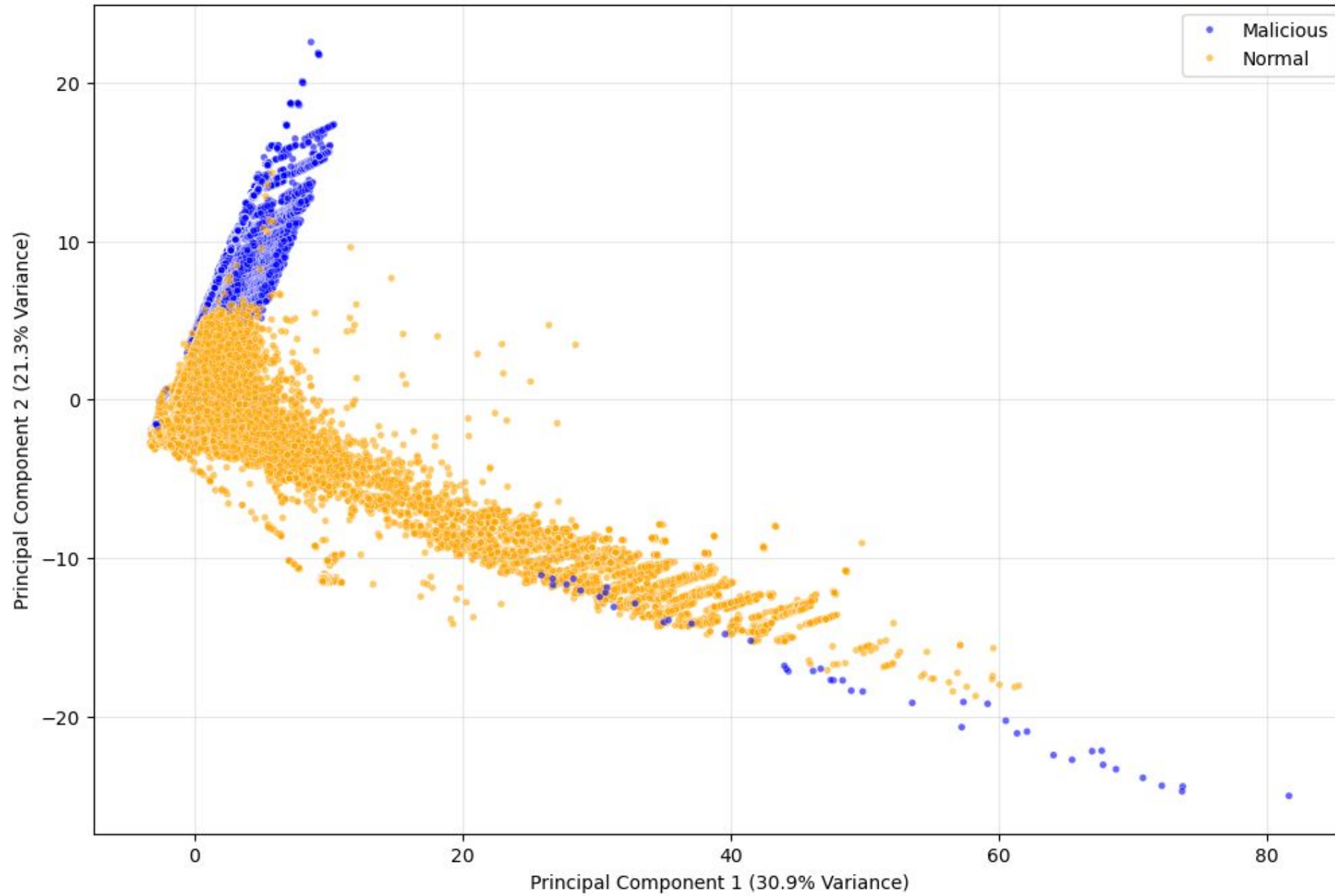
```
Recall: 89.018%
```

```
Threshold: 2.000%
```





PCA of Network Traffic



Interpretation, Conclusion, and Future Work

PCA graph shows that network traffic is complex; malicious connections overlap benign ones:

- Supervised tree models did well at finding these hidden attacks
- Unsupervised models struggled a lot with chronological drift
- Implementing continuous learning to adjust to changing network conditions
- Creating an ensemble pipeline where SVM flags anomalies and Random Forest categorizes them.

